Department of
Computer Science and Engineering

Title: Semaphore

Operating System Lab
CSE 310

Green University of Bangladesh

1 Objective(s)

- To gather knowledge about semaphore.
- To learn about the implementation of binary and counting semaphore.

2 Semaphore

Semaphore is something like a protected variable. It provides a means to restrict access to a resource that is shared amongst two or more processes.There are two operations :

- **Acquire:** The acquire operation allows a process to take the semaphore, and if it has already been acquired, then the process blocks until its available.
- **Release:** If a process has the semaphore, it can release it, which allows other processes to acquire it.

2.1 Types Of Semaphore

Semaphores come in two basic varieties. They are :

- **Binary semaphores:** The binary semaphore represents a single resource; therefore, when one process has acquired it, others are blocked until it is released. As shown in Figure 1, process A performs the

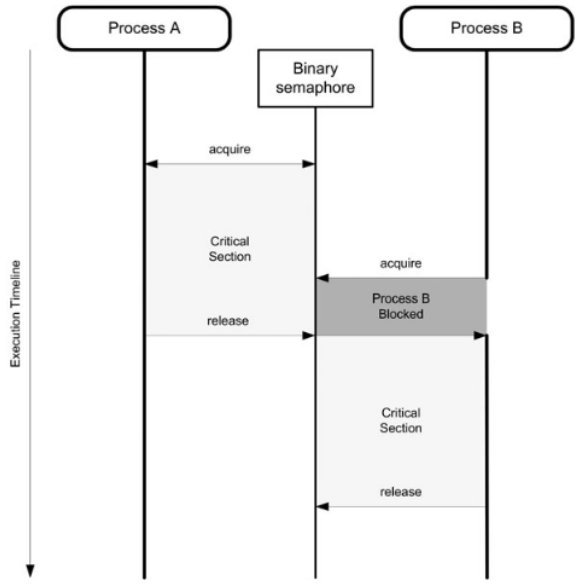

Figure 1: Binary semaphore example with two processes.

acquire first and, therefore, is provided with the semaphore. The period in which the semaphore is owned by the process is commonly called a critical section that can only be carried out by one process. While process A has the semaphore, process B is not permitted to perform its critical section.

- **Counting semaphores:** The counting semaphore is used to represent shared resources in quantities greater than one. Consider a pool of buffers. A counting semaphore can represent the entire set of buffers by setting its value to the number of buffers available. Each time a process requires a buffer, it acquires the semaphore, which decrements its value. When the semaphore value reaches zero, processes are blocked until the value becomes nonzero. When a semaphore is released, the semaphore value is increased, thus permitting other processes to acquire a semaphore (and associated buffer). This is the one use for a counting semaphore (see Figure 2).

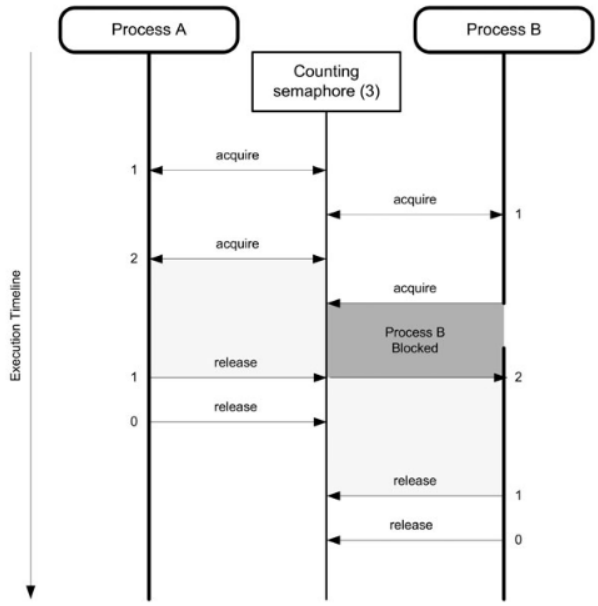

Figure 2: Counting semaphore example with two processes.

2.2 Creating A Semaphore

To create a semaphore, we need to use the semget API function. This function takes a semaphore key, a semaphore count, and a set of flags. The count represents the number of semaphores in the set. At line 10, we use the semget with your semaphore key, semaphore set count, and command (with read/write permissions).

API Functions	Uses
semget	Create a new semaphore.
	Get an existing semaphore.
semop	Acquire or release a semaphore.
semctl	Get info about a semaphore.
	Set info about a semaphore.
	Remove a semaphore.

Table 1: Semaphore API Functions and Their Uses

2.2.1 Implementation in C

```
1  //Implementation of common.h header file
2
3  #define MAX_LINE 80
4  #define MY_SEM_ID 111
5  typedef struct {
6  long type; // Msg Type (> 0)
7  float fval; // User Message
8  unsigned int uival; // User Message
9  char strval[MAX_LINE+1]; // User Message
10  }MY_TYPE_T;
```

```
1  #include <stdio.h>
2  #include <sys/sem.h>
3  #include "common.h"
4  int main()
5  {
```

```

int semid;

/* Create the semaphore with the id MY_SEM_ID */
semid = semget( MY_SEM_ID, 1, 0666 | IPC_CREAT );

if (semid >= 0) {

printf( "semcreate: Created a semaphore %d\n", semid );
}
}
```

2.3 Getting A Semaphore

Recall that your previously created semaphore (in code2.2.1) was initialized with a value of zero. This is identical to a binary semaphore already having been acquired. The following code 2.3.1 illustrates an application acquiring your semaphore.

2.3.1 Implementation in C

```
#include <stdio.h>
#include <sys/sem.h>
#include <stdlib.h>
#include "common.h"

int main()
{
int semid;
struct sembuf sb;

/* Get the semaphore with the id MY_SEM_ID */
semid = semget( MY_SEM_ID, 1, 0666 | IPC_CREAT );

if (semid >= 0) {

sb.sem_num = 0;
sb.sem_op = -1;
sb.sem_flg = 0;

printf( "semacq: Attempting to acquire semaphore
%d\n", semid );
/* Acquire the semaphore */
if ( semop( semid, &sb, 1 ) == -1 ) {

printf( "semacq: semop failed.\n" );
exit(-1);

}

printf( "semacq: Semaphore acquired %d\n", semid );

}

return 0;
}
```

2.4 Releasing A Semaphore

Now its time to look at a release example. This example demonstrates the semop API function from the perspective of releasing the semaphore (see following code 2.4.1).

2.4.1 Implementation in C

```c
#include <stdio.h>
#include <sys/sem.h>
#include <stdlib.h>
#include "common.h"

int main()
{
int semid;
struct sembuf sb;

/* Get the semaphore with the id MY_SEM_ID */
semid = semget( MY_SEM_ID, 1, 0666 | IPC_CREAT );

if (semid >= 0) {

printf( "semrel: Releasing semaphore %d\n", semid );

sb.sem_num = 0;
sb.sem_op = 1;
sb.sem_flg = 0;

/* Release the semaphore */
if (semop( semid, &sb, 1 ) == -1) {

printf("semrel: semop failed.\n");
exit(-1);

}

printf( "semrel: Semaphore released %d\n", semid );

}

return 0;
}
```

2.5 Retrieving the Current Semaphore Count

The following code 2.5.1, demonstrates reading the current value of the semaphore. You achieve this using the semctl API function

2.5.1 Implementation in C

```c
#include <stdio.h>
#include <sys/sem.h>
#include <stdlib.h>
#include "common.h"

int main()
{
int semid, cnt;
```

```

/* Get the semaphore with the id MY_SEM_ID */
semid = semget( MY_SEM_ID, 1, 0666 | IPC_CREAT );

if (semid >= 0) {

/* Read the current semaphore count */
cnt = semctl( semid, 0, GETVAL );

if (cnt != -1) {

printf("semcrd: current semaphore count %d.\n", cnt);

}

}

return 0;
}
```

2.6 Setting the Current Semaphore Count

The following code 2.6.1, demonstrates setting the current value of the semaphore.

2.6.1 Implementation in C

```
#include <stdio.h>
#include <sys/sem.h>
#include <stdlib.h>
#include "common.h"

int main()
{
int semid, ret;

/* Get the semaphore with the id MY_SEM_ID */
semid = semget( MY_SEM_ID, 1, 0666 | IPC_CREAT );

if (semid >= 0) {

/* Read the current semaphore count */
ret = semctl( semid, 0, SETVAL, 6 );

if (ret != -1) {

printf( "semcrd: semaphore count updated.\n" );

}

}

return 0;
}
```

2.7 Removing a Semaphore

Removing a semaphore is also performed through the semctl API function. After retrieving the semphore identifier (line 10 in following code2.7.1), you remove the semaphore using the semctl API function and the IPC_RMID command (at line 14).

2.7.1 Implementation in C

```
1  #include <stdio.h>
2  #include <sys/sem.h>
3  #include "common.h"
4
5  int main()
6  {
7  int semid, ret;
8
9  /* Get the semaphore with the id MY_SEM_ID */
10 semid = semget( MY_SEM_ID, 1, 0666 | IPC_CREAT );
11
12 if (semid >= 0) {
13
14 ret = semctl( semid, 0, IPC_RMID);
15
16 if (ret != -1) {
17
18 printf( "Semaphore %d removed.\n", semid );
19
20 }
21
22 }
23
24 return 0;
25 }
```

3 Lab Task (Please implement yourself and show the output to the instructor)

- Create a semaphore and try to update its count value.
- Destroy that semaphore

4 Lab Exercise (Submit as a report)

Suppose we have a buffer that is of fixed size, and the producer will produce items and place them in the buffer. The consumer can pick items from the buffer and consume them. Our task is to ensure that when the item is placed in the buffer by the producer, the consumer should not consume it at the same time the producer produces and places an item into the buffer. The critical section here is the buffer.

- **Input:**

```
Buffer size: 3 (fixed size)
Number of producers: 2
Number of consumers: 2
```

- **Output:**

```
--- Producer 1 produces item 1 and places it in the buffer ---
Buffer: [1, -, -]
--- Producer 2 produces item 2 and places it in the buffer ---
Buffer: [1, 2, -]
--- Producer 1 produces item 3 and places it in the buffer ---
Buffer: [1, 2, 3]
--- Consumer 1 consumes item 1 from the buffer ---
Buffer: [-, 2, 3]
--- Consumer 2 consumes item 2 from the buffer ---
Buffer: [-, -, 3]
--- Producer 2 produces item 4 and places it in the buffer ---
Buffer: [4, -, 3]
--- Consumer 1 consumes item 3 from the buffer ---
Buffer: [4, -, -]
--- Consumer 2 consumes item 4 from the buffer ---
Buffer: [-, -, -]
```

5 Discussion & Conclusion

Based on the focused objective(s) to understand about the semaphore using c program, the additional lab exercise made me more confident towards the fulfilment of the objectives(s).

6 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be deducted if any such copying is detected.